UNIT 03: QUANTUM PROGRAMMING PLATFORMS

Table of Contents

INTRODUCTION

What if programming a quantum computer — something that existed only in physics laboratories a decade ago — were now as simple as writing a few lines of Python on your laptop?

On 4 May 2016, IBM did something unprecedented: it connected a 5-qubit quantum processor at its T.J. Watson Research Centre to the cloud and opened it to anyone with an internet connection. Within two weeks, 17,000 people had logged in to run circuits on a real quantum machine. Today, over 600,000 users across 700 universities program IBM's quantum hardware through Qiskit — an open-source framework that reached its stable 1.0 release in February 2024. Google's Cirq and Xanadu's PennyLane followed similar paths, creating an ecosystem where three powerful, free frameworks compete to make quantum programming accessible.

Behind this accessibility lie three capabilities this unit teaches: **circuit construction** (building quantum programs from gates and measurements), **framework architecture** (understanding how software translates your code into hardware pulses), and **data encoding** (mapping classical information into quantum states — the critical bridge between classical data and quantum computation). Choosing the right framework and encoding strategy can mean the difference between a quantum program that runs in seconds and one that never converges.

This unit gives you hands-on command of all three major frameworks — Qiskit, PennyLane, and Cirq — so you can move from understanding quantum theory to writing and executing quantum code.

Before studying this unit, you should have:

- Completion of the Quantum Computing Fundamentals unit
- Understanding of quantum gates (H, X, Y, Z, CNOT) and their matrix representations
- Familiarity with Bell states and quantum measurement
- Basic Python programming skills

LEARNING OBJECTIVES

Upon successful completion of this unit, learners will be able to:

1. Implement quantum circuits in Qiskit using QuantumCircuit, execute circuits on simulators, and interpret measurement results from quantum programs
2. Apply the PennyLane framework for QML by implementing data encoding methods (Basis, Amplitude, and Angle embedding) and utilising automatic differentiation capabilities
3. Implement quantum circuits in Cirq framework using Moment-based structure with LineQubit and GridQubit for different qubit topologies
4. Evaluate and compare quantum programming frameworks (Qiskit, PennyLane, Cirq) to justify framework selection for specific quantum computing tasks
5. Design data encoding strategies for quantum circuits by analysing trade-offs between encoding methods for different data types and qubit requirements

3.1 QISKIT FRAMEWORK

3.1.1 INTRODUCTION TO QISKIT

Qiskit (Quantum Information Software Kit) is IBM's open-source software development kit for working with quantum computers. It provides tools for creating and manipulating quantum programs at the level of pulses, circuits, and application modules. Among the three frameworks covered in this unit, Qiskit is the most comprehensive ecosystem — spanning from low-level circuit construction to high-level algorithm libraries — and is widely regarded as the best starting point for learning quantum computing due to its extensive tutorials and documentation.

Qiskit is designed to be:

- **Accessible**: Python-based with intuitive syntax
- **Comprehensive**: Covers circuit design to algorithm implementation
- **Hardware-connected**: Direct access to IBM Quantum systems

- **Education-focused**: Extensive tutorials and an open-source textbook

To install Qiskit, use pip:

```
pip install qiskit
```

The framework supports execution on both classical simulators and real quantum hardware through IBM Quantum Experience. IBM announced its 1,121-qubit Condor processor in December 2023, making Qiskit the primary gateway to some of the world's most powerful quantum hardware.

> ***Did You Know?***
>
> *IBM's Qiskit is one of the most widely adopted quantum computing frameworks, with over 550,000 registered users and nearly 2,900 research papers citing the platform. Its open-source community has contributed to making quantum computing accessible to researchers and students worldwide.*

3.1.2 QISKIT ARCHITECTURE

Qiskit consists of four main components, each serving a distinct purpose:

Terra (Earth): The foundation layer that provides the basic building blocks for composing quantum programs at the circuit level. Terra handles:

- Quantum circuit construction
- Circuit optimisation and transpilation
- Backend communication

Aer (Air): A high-performance quantum simulator framework. Aer provides:

- State vector simulation for exact results
- QASM simulation for shot-based measurement
- Noise models for realistic simulation

Ignis (Fire): A framework for understanding and mitigating noise in quantum systems. Ignis includes:

- Error characterisation tools

- Error mitigation techniques
- Verification protocols

Aqua (Water): A library of algorithms for various quantum applications:

- Chemistry and physics simulations
- Optimisation algorithms
- Machine learning algorithms
- Finance applications

Component	Purpose	Key Feature
Terra	Circuit construction	Transpilation and optimisation
Aer	Simulation	Noise models for realistic testing
Ignis	Error handling	Error characterisation and mitigation
Aqua	Applications	Algorithm libraries for ML, chemistry

Table 1: Qiskit Architecture Components and Their Roles.

Note: The Terra/Aer/Ignis/Aqua naming reflects Qiskit's historical architecture. Since Qiskit 1.0 (February 2024), the framework has been consolidated into a single `qiskit` package. Aqua was deprecated in 2021 (its algorithms moved to dedicated application modules like `qiskit-machine-learning`), and Ignis was replaced by `qiskit-experiments`. Aer continues as a separate package (`qiskit-aer`). Understanding the original four-component model remains useful for reading older tutorials and research papers.

This modular architecture allows developers to use only the components they need while maintaining interoperability. For quantum machine learning, the core `qiskit` package handles circuit building and transpilation, while `qiskit-aer` provides simulation and `qiskit-machine-learning` offers ready-made ML algorithm implementations.

3.1.3 CREATING QUANTUM CIRCUITS

The fundamental object in Qiskit is the `QuantumCircuit` class, which represents a quantum circuit as a sequence of gates and measurements.

Basic Circuit Creation:

```
from qiskit import QuantumCircuit, QuantumRegister,
ClassicalRegister

# Method 1: Simple creation with qubit and classical
bit count
qc = QuantumCircuit(2, 2)  # 2 qubits, 2 classical
bits

# Method 2: Using explicit registers
qr = QuantumRegister(2, 'q')    # 2 qubits named 'q'
cr = ClassicalRegister(2, 'c')   # 2 classical bits
named 'c'
qc = QuantumCircuit(qr, cr)
```

The `QuantumRegister` holds quantum bits (qubits), while the `ClassicalRegister` stores classical bits for measurement results. You need both because quantum measurement collapses a qubit's state into a classical 0 or 1, and that classical result must be stored somewhere.

> ***Common Misconception***
>
> *Many beginners wonder why classical bits are needed at all. The reason is that quantum information cannot be directly read — it must be measured first, which produces a classical outcome. The ClassicalRegister is where these measurement outcomes are stored.*

Visualising Circuits:

```
# Text-based visualisation
print(qc.draw())

# Matplotlib visualisation (requires matplotlib)
qc.draw(output='mpl')
```

3.1.4 QUANTUM GATES IN QISKIT

Qiskit provides methods for all standard quantum gates. Gates are applied to circuits using method calls on the QuantumCircuit object.

Single-Qubit Gates:

```
qc = QuantumCircuit(1)

qc.h(0)       # Hadamard gate on qubit 0
qc.x(0)       # Pauli-X (NOT) gate
qc.y(0)       # Pauli-Y gate
qc.z(0)       # Pauli-Z gate
```

Rotation Gates:

```
import numpy as np

qc.rx(np.pi/4, 0)    # Rotation around X-axis by pi/4
qc.ry(np.pi/2, 0)    # Rotation around Y-axis by pi/2
qc.rz(np.pi, 0)      # Rotation around Z-axis by pi
```

Rotation gates are particularly important for quantum machine learning because they allow continuous parameterisation of quantum states. When you train a quantum model, the rotation angles become the trainable parameters — analogous to weights in a classical neural network.

Multi-Qubit Gates:

```
qc = QuantumCircuit(2)

qc.cx(0, 1)      # CNOT: control=0, target=1
qc.cz(0, 1)      # Controlled-Z
qc.swap(0, 1)    # SWAP gate
```

Measurements:

```
qc.measure(0, 0)       # Measure qubit 0, store in
classical bit 0
qc.measure([0, 1], [0, 1])   # Measure multiple qubits
```

3.1.5 CIRCUIT EXECUTION AND SIMULATION

To execute a quantum circuit, you need to select a backend and use the `execute()` function.

Selecting a Simulator Backend:

```
from qiskit import Aer

# Available simulators in Aer:
# - qasm_simulator: Shot-based simulation with
measurements
# - statevector_simulator: Returns full quantum state
# - unitary_simulator: Returns unitary matrix of
circuit

backend = Aer.get_backend('qasm_simulator')
```

Executing the Circuit:

```
from qiskit import execute

# Execute circuit with 1000 shots (measurement
repetitions)
job = execute(qc, backend, shots=1000)

# Wait for job completion and get results
result = job.result()
```

The `shots` parameter controls how many times the circuit is executed and measured. Since quantum measurement is probabilistic, running multiple shots produces a histogram of outcomes that approximates the theoretical probability distribution. Higher shot counts give more accurate statistics but take longer to run.

Complete Bell State Example:

```
from qiskit import QuantumCircuit, execute, Aer

# Create Bell state circuit
qc = QuantumCircuit(2, 2)
qc.h(0)           # Put qubit 0 in superposition
```

```
qc.cx(0, 1)        # Entangle qubit 1 with qubit 0
qc.measure([0, 1], [0, 1])

# Execute on simulator
backend = Aer.get_backend('qasm_simulator')
job = execute(qc, backend, shots=1000)
result = job.result()

# Get measurement counts
counts = result.get_counts()
print(counts)  # Output: {'00': ~500, '11': ~500}
```

3.1.6 INTERPRETING RESULTS

The `result.get_counts()` method returns a dictionary mapping measurement outcomes to their occurrence counts.

```
counts = result.get_counts()
# Example output: {'00': 498, '11': 502}
```

Understanding the Output:

- Keys are bit strings representing measurement outcomes
- Values are the number of times each outcome occurred
- In Qiskit, bit strings are ordered from right to left (qubit 0 is rightmost)

  Common Pitfall

  Qiskit's bit ordering is right-to-left, which is the opposite of what most beginners expect. In a result like '01', qubit 0 is the rightmost bit (1) and qubit 1 is the leftmost bit (0). This convention follows the standard mathematical notation for tensor products but can cause confusion when reading results.

Visualisation:

```
from qiskit.visualization import plot_histogram

plot_histogram(counts)
```

This creates a bar chart showing the probability distribution of measurement outcomes. For the Bell state above, you would see approximately equal bars for '00' and '11', confirming the entanglement.

Activity

Install Qiskit in your Python environment using `pip install qiskit`*. Create a Bell state circuit following the example in Section 3.1.5, execute it with 1000 shots, and verify that you observe only the '00' and '11' outcomes. Then modify the circuit to create the Bell state* $|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$ *by adding a Pauli-X gate on qubit 1 after the Hadamard (on qubit 0) and before the CNOT.*

3.2 PENNYLANE FRAMEWORK

3.2.1 INTRODUCTION TO PENNYLANE

PennyLane is an open-source software framework for quantum machine learning, quantum computing, and quantum chemistry developed by Xanadu. Unlike Qiskit and Cirq, which are general-purpose quantum computing frameworks, PennyLane is specifically designed with machine learning in mind. Its distinguishing feature is built-in automatic differentiation of quantum circuits, which enables gradient-based optimisation of variational circuits — the core building block of quantum machine learning algorithms.

Key features of PennyLane include:

- **Automatic differentiation**: Compute gradients of quantum circuits using the parameter-shift rule
- **ML library integration**: Works natively with PyTorch, TensorFlow, and JAX
- **Hardware agnostic**: Runs on multiple quantum platforms including IBM, Google, and Xanadu hardware
- **QML-focused**: Built-in support for variational algorithms and data encoding

To install PennyLane:

```
pip install pennylane
```

Think About It

Why is automatic differentiation so important for quantum machine learning? Consider how classical neural networks are trained: gradient descent requires computing derivatives of the loss function with respect to all weights. PennyLane brings this same capability to quantum circuits, making it possible to train quantum models using the same optimisation techniques that power classical deep learning.

3.2.2 DEVICES AND BACKENDS

In PennyLane, quantum computations are executed on **devices**. A device represents either a simulator or actual quantum hardware.

Creating a Device:

```
import pennylane as qml

# Create a device with N qubits
n_qubits = 4
dev = qml.device("default.qubit", wires=n_qubits)
```

The `wires` parameter specifies the number of qubits (also called wires in PennyLane terminology). This naming convention reflects PennyLane's circuit-centric design where qubits are thought of as wires carrying quantum information.

Available Devices:

Device	Description
`default.qubit`	PennyLane's built-in pure-state simulator
`lightning.qubit`	High-performance C++ simulator
`qiskit.aer`	IBM's Aer simulator (requires plugin)
`cirq.simulator`	Google's Cirq simulator (requires plugin)

Table 2: PennyLane Available Devices and Their Descriptions.

The ability to swap devices without changing circuit code is a major advantage. You can develop and test on `default.qubit`, then switch to `lightning.qubit` for faster simulation, or to a real hardware device — all without modifying your circuit.

3.2.3 QNODE PATTERN

The core abstraction in PennyLane is the **QNode** (quantum node). A QNode is a Python function decorated with `@qml.qnode` that contains quantum operations and returns measurement results. This design makes quantum circuits feel like ordinary Python functions, which is why PennyLane integrates so seamlessly with ML libraries.

Basic QNode Structure:

```
import pennylane as qml

dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev)
def my_circuit(params):
    # Quantum operations
    qml.RX(params[0], wires=0)
    qml.RY(params[1], wires=1)
    qml.CNOT(wires=[0, 1])

    # Return measurement
    return qml.expval(qml.PauliZ(0))
```

Key Points:

- The decorator `@qml.qnode(dev)` binds the function to a device
- Parameters can be passed to the function for variational circuits
- The function must return a measurement

Calling a QNode:

```
import numpy as np

# Execute the circuit with specific parameters
result = my_circuit([0.1, 0.2])
print(result)  # Returns expectation value
```

Because a QNode behaves like a regular Python function, you can pass it to optimisers, compute gradients, and use it inside larger computational graphs — exactly as you would with any differentiable function in PyTorch or TensorFlow.

3.2.4 QUANTUM OPERATIONS IN PENNYLANE

PennyLane provides a comprehensive library of quantum operations.

Single-Qubit Gates:

```
qml.Hadamard(wires=0)       # Hadamard gate
qml.PauliX(wires=0)         # Pauli-X (NOT)
qml.PauliY(wires=0)         # Pauli-Y
qml.PauliZ(wires=0)         # Pauli-Z
```

Rotation Gates:

```
qml.RX(theta, wires=0)      # X-rotation by angle theta
qml.RY(theta, wires=0)      # Y-rotation
qml.RZ(theta, wires=0)      # Z-rotation
```

Multi-Qubit Gates:

```
qml.CNOT(wires=[0, 1])      # Control=0, target=1
qml.CZ(wires=[0, 1])        # Controlled-Z
```

Bell State in PennyLane:

```
dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev)
def bell_state():
    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 1])
    return qml.state()

state = bell_state()
print(state)  # [0.707+0j, 0, 0, 0.707+0j]
```

Notice how PennyLane's Bell state implementation is more concise than Qiskit's. There are no explicit classical registers or measurement commands — the return statement

specifies what information to extract. This reflects PennyLane's philosophy of keeping quantum programming as close to standard Python as possible.

3.2.5 DATA ENCODING METHODS

One of PennyLane's greatest strengths is its built-in support for encoding classical data into quantum states. This is fundamental to quantum machine learning — before any quantum algorithm can process classical data, that data must be embedded into a quantum state.

Basis Embedding:

Basis embedding encodes a classical integer as a computational basis state.

```
@qml.qnode(dev)
def basis_embedding_circuit(x):
    qml.BasisEmbedding(x, wires=range(3))
    return qml.state()

# Example: Encode integer 5 (binary 101) into 3 qubits
state = basis_embedding_circuit(5)
# Result: |101>
```

The formula for basis embedding:

$$|x\rangle = |x_1 x_2 \dots x_n\rangle$$

How to read: "Ket x equals ket x-sub-1 x-sub-2 through x-sub-n" — the classical integer x is represented by its binary digits as a quantum computational basis state.

Where:

- $|x\rangle$ = quantum state encoding integer x
- $x_1, x_2, \dots, x_n$ = binary digits of x
- n = number of qubits

Amplitude Embedding:

Amplitude embedding encodes a normalised vector as quantum state amplitudes, achieving exponential data compression.

```
@qml.qnode(dev)
def amplitude_embedding_circuit(features):
    qml.AmplitudeEmbedding(features, wires=range(2),
normalize=True)
    return qml.state()

# Requires 2^n values for n qubits
features = [0.5, 0.5, 0.5, 0.5]  # 4 values for 2
qubits
state = amplitude_embedding_circuit(features)
```

The formula for amplitude embedding:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \hat{x}_i |i\rangle$$

How to read: "Psi equals the sum from i=0 to 2-to-the-n minus 1 of x-hat-sub-i times ket i" — the quantum state is a superposition where each amplitude equals the corresponding normalised feature value.

Where:

- $|\psi\rangle$ = encoded quantum state
- $\hat{x}_i$ = i-th component of the normalised feature vector ($\|\hat{x}\| = 1$)
- $|i\rangle$ = computational basis state
- n = number of qubits

This is particularly powerful: a vector of 2^n features requires only n qubits. For example, 1024 features need just 10 qubits — an exponential compression that classical computing cannot match.

Angle Embedding:

Angle embedding encodes each feature as a rotation angle on a separate qubit.

```
@qml.qnode(dev)
def angle_embedding_circuit(features):
    qml.AngleEmbedding(features, wires=range(3),
rotation='Y')
    return qml.state()

# One feature per qubit
```

```
features = [0.1, 0.2, 0.3]
state = angle_embedding_circuit(features)
```

The formula for angle embedding:

$$|\psi\rangle = \otimes_{j=1}^{n} R_Y(x_j)|0\rangle$$

How to read: "Psi equals the tensor product from j=1 to n of R-Y of x-sub-j applied to ket zero" — each qubit is independently rotated around the Y-axis by an angle equal to the corresponding feature value.

Where:

- $|\psi\rangle$ = encoded quantum state
- $\otimes$ = tensor product (each qubit handled independently)
- $R_Y(x_j)$ = Y-rotation gate with angle x_j
- x_j = j-th feature value
- $|0\rangle$ = initial qubit state
- n = number of features (= number of qubits, since each feature uses one qubit)

Choosing an Encoding Method:

Method	Qubits Needed	Best For	Trade-off
Basis	n (for n -bit integers)	Integer/categorical data	Limited to discrete values
Amplitude	$\log_2(N)$ (for N features)	Large feature vectors	Complex state preparation circuit
Angle	1 per feature	Small feature sets	Linear qubit scaling

Table 3: Comparison of Data Encoding Methods in PennyLane.

Tips and Tricks

When choosing an encoding method for a QML task, consider the size of your feature vectors. For datasets with hundreds or thousands of features (common in real-world ML), amplitude embedding is often the best choice because it compresses exponentially. For small datasets with fewer than

20 features, angle embedding is simpler to implement and avoids the complexity of amplitude state preparation.

3.2.6 MEASUREMENTS IN PENNYLANE

PennyLane supports several types of measurements as return values from QNodes.

Expectation Value:

```
return qml.expval(qml.PauliZ(0))
```

Returns the expected value of the observable, a scalar between -1 and 1 for Pauli operators. This is the most common measurement type in QML.

Variance:

```
return qml.var(qml.PauliZ(0))
```

Returns the variance of the measurement outcomes.

Probability Distribution:

```
return qml.probs(wires=[0, 1])
```

Returns the probabilities of all computational basis states.

State Vector:

```
return qml.state()
```

Returns the full quantum state vector (only available on simulators).

Sample Measurements:

```
return qml.sample(qml.PauliZ(0))
```

Returns individual measurement samples.

3.2.7 VARIATIONAL CIRCUITS PREVIEW

PennyLane excels at variational quantum circuits — circuits with trainable parameters that are optimised using gradient descent.

```
@qml.qnode(dev)
def variational_circuit(weights, x):
    # Data encoding layer
    qml.AngleEmbedding(x, wires=range(n_qubits))

    # Variational layer with trainable weights
    for i in range(n_qubits):
        qml.RY(weights[i], wires=i)

    # Entangling layer
    for i in range(n_qubits - 1):
        qml.CNOT(wires=[i, i+1])

    return qml.expval(qml.PauliZ(0))
```

The key advantage is that PennyLane can compute gradients automatically using the parameter-shift rule:

```
# Compute gradient with respect to weights
grad_fn = qml.grad(variational_circuit, argnum=0)
gradient = grad_fn(weights, x)
```

This enables optimisation using gradient descent, which is explored in detail in later units on Quantum Neural Networks.

Common Misconception

Students sometimes assume that quantum gradients require the same backpropagation algorithm used in classical neural networks. In reality, PennyLane uses the ***parameter-shift rule****, a quantum-specific technique that evaluates the circuit at slightly shifted parameter values to estimate gradients. This approach is mathematically exact (not an approximation) and works on real quantum hardware, unlike backpropagation which requires access to intermediate state values.*

3.3 CIRQ FRAMEWORK

3.3.1 INTRODUCTION TO CIRQ

Cirq is a Python framework for creating, editing, and invoking Noisy Intermediate Scale Quantum (NISQ) circuits. Developed by Google, Cirq is designed with a focus on near-term quantum devices and their constraints. While Qiskit prioritises accessibility and PennyLane prioritises ML integration, Cirq prioritises fine-grained, hardware-aware control over quantum circuits. It was the framework used in Google's landmark quantum supremacy experiment in 2019.

Key characteristics of Cirq:

- **NISQ-focused**: Designed for current noisy quantum devices
- **Hardware-aware**: Considers device topology and connectivity
- **Moment-based**: Operations are organised into time slices
- **Research-oriented**: Used in quantum supremacy experiments
- **Flexible**: Fine-grained control over circuit construction

To install Cirq:

```
pip install cirq
```

3.3.2 QUBITS IN CIRQ

Cirq uses explicit qubit objects rather than integer indices. This design decision reflects Cirq's hardware-aware philosophy — on a real quantum chip, qubits have physical locations and not all qubits can interact directly.

LineQubit:

```
import cirq

# Create individual qubits
q0 = cirq.LineQubit(0)
q1 = cirq.LineQubit(1)
```

```
# Create a range of qubits
q0, q1, q2 = cirq.LineQubit.range(3)
```

LineQubits are arranged in a linear topology, suitable for general-purpose simulation and devices with one-dimensional qubit connectivity.

GridQubit:

```
# Create qubits in a 2D grid
q00 = cirq.GridQubit(0, 0)
q01 = cirq.GridQubit(0, 1)
q10 = cirq.GridQubit(1, 0)

# Create a grid of qubits
qubits = cirq.GridQubit.rect(2, 3)  # 2x3 grid
```

GridQubits model the layout of superconducting quantum chips like Google's Sycamore processor, which arranges qubits in a two-dimensional grid. Using GridQubits ensures your circuit respects the physical connectivity constraints of the hardware.

Did You Know?

Google's Sycamore processor, which demonstrated quantum supremacy in 2019, uses a grid of 53 superconducting qubits arranged in a pattern that Cirq's GridQubit class directly models. This tight coupling between software and hardware is what makes Cirq the preferred framework for Google's quantum research.

3.3.3 BUILDING CIRCUITS

Cirq circuits are built from operations organised into **Moments** — a concept unique to Cirq among the three frameworks.

Basic Circuit Creation:

```
import cirq

q0, q1 = cirq.LineQubit.range(2)

# Method 1: Using list of operations
```

```
circuit = cirq.Circuit([
    cirq.H(q0),
    cirq.CNOT(q0, q1),
    cirq.measure(q0, q1, key='result')
])

# Method 2: Using append
circuit = cirq.Circuit()
circuit.append(cirq.H(q0))
circuit.append(cirq.CNOT(q0, q1))
circuit.append(cirq.measure(q0, q1, key='result'))
```

Understanding Moments:

A Moment is a collection of operations that execute simultaneously (on different qubits). Cirq automatically organises operations into Moments, which gives you explicit visibility into the parallelism of your circuit:

```
circuit = cirq.Circuit([
    cirq.H(q0),           # Moment 0
    cirq.H(q1),           # Also Moment 0 (different
qubit)
    cirq.CNOT(q0, q1),    # Moment 1 (depends on both
qubits)
    cirq.measure(q0, q1)  # Moment 2
])

print(circuit)
```

This moment-based structure differs from Qiskit's sequential approach. In Qiskit, gates are appended one at a time and the framework handles scheduling internally. In Cirq, the grouping into moments is explicit, giving researchers direct control over operation timing — crucial for optimising circuits on real hardware where gate execution time matters.

3.3.4 GATES AND OPERATIONS

Cirq provides a comprehensive set of quantum gates.

Single-Qubit Gates:

```
cirq.H(q0)       # Hadamard
cirq.X(q0)       # Pauli-X
cirq.Y(q0)       # Pauli-Y
cirq.Z(q0)       # Pauli-Z
```

Rotation Gates:

```
import numpy as np

cirq.rx(np.pi/4)(q0)     # X-rotation
cirq.ry(np.pi/2)(q0)     # Y-rotation
cirq.rz(np.pi)(q0)       # Z-rotation
```

Multi-Qubit Gates:

```
cirq.CNOT(q0, q1)        # Controlled-NOT
cirq.CZ(q0, q1)          # Controlled-Z
cirq.SWAP(q0, q1)        # SWAP
```

Power Notation:

Cirq supports fractional gate powers using the `**` operator, which is a distinctive syntactic feature:

```
cirq.X(q0)**0.5      # Square root of X
cirq.Z(q0)**0.25     # T gate (applies pi/4 phase
shift)
```

This power notation is unique to Cirq and provides an elegant way to express fractional rotations that would require explicit angle calculations in other frameworks.

3.3.5 MEASUREMENTS

Measurements in Cirq use the `cirq.measure()` function with a key for accessing results.

```
# Measure single qubit
cirq.measure(q0, key='m0')

# Measure multiple qubits with one key
cirq.measure(q0, q1, key='result')
```

```
# Measure with separate keys
[cirq.measure(q0, key='a'), cirq.measure(q1, key='b')]
```

The `key` parameter names the measurement result, which is used to access outcomes after simulation. This named-key approach is more explicit than Qiskit's index-based classical register system and makes it easier to track multiple measurements in complex circuits.

3.3.6 SIMULATION AND EXECUTION

Cirq provides the `cirq.Simulator` class for executing circuits.

Basic Simulation:

```
simulator = cirq.Simulator()

# Run circuit with specified repetitions
result = simulator.run(circuit, repetitions=1000)

# Access measurement results
print(result.histogram(key='result'))
```

Complete Bell State Example:

```
import cirq

# Create qubits
q0, q1 = cirq.LineQubit.range(2)

# Build Bell state circuit
circuit = cirq.Circuit([
    cirq.H(q0),
    cirq.CNOT(q0, q1),
    cirq.measure(q0, q1, key='result')
])

print(circuit)

# Simulate
simulator = cirq.Simulator()
```

```
result = simulator.run(circuit, repetitions=1000)

# Show results
print(result.histogram(key='result'))
# Output: Counter({0: ~500, 3: ~500})
# Note: 0 = 00 (binary), 3 = 11 (binary)
```

Common Pitfall

Cirq returns measurement results as integers rather than bit strings. In the Bell state example, '00' appears as 0 and '11' appears as 3 (since binary 11 = decimal 3). This can be confusing if you expect string-based results like Qiskit provides. Use Python's `bin()` function to convert if needed: `bin(3)` gives `'0b11'`.

3.3.7 QUANTUM TELEPORTATION EXAMPLE

Let us implement the quantum teleportation protocol in Cirq, demonstrating a practical application of the framework with a more complex circuit.

```
import cirq
import numpy as np

def quantum_teleportation():
    # Create three qubits: message, alice, bob
    msg, alice, bob = cirq.LineQubit.range(3)

    circuit = cirq.Circuit()

    # Step 1: Prepare message qubit in an arbitrary
state
    # Using X^0.25 as an example state to teleport
    circuit.append(cirq.X(msg)**0.25)

    # Step 2: Create Bell pair between Alice and Bob
    circuit.append(cirq.H(alice))
    circuit.append(cirq.CNOT(alice, bob))

    # Step 3: Alice's operations (Bell measurement)
```

```
    circuit.append(cirq.CNOT(msg, alice))
    circuit.append(cirq.H(msg))

    # Step 4: Measurements
    circuit.append(cirq.measure(msg, key='m1'))
    circuit.append(cirq.measure(alice, key='m2'))

    # Step 5: Bob's corrections (classically
controlled)
    circuit.append(cirq.CNOT(alice, bob))
    circuit.append(cirq.CZ(msg, bob))

    return circuit

# Create and display circuit
teleport_circuit = quantum_teleportation()
print(teleport_circuit)

# Simulate
simulator = cirq.Simulator()
result = simulator.run(teleport_circuit,
repetitions=100)
print(result)
```

This example demonstrates creating multiple qubits, applying single and multi-qubit gates, measurement with multiple keys, and classical control operations.

3.3.8 CHOOSING BETWEEN FRAMEWORKS

Having now studied all three frameworks, you can make informed decisions about which to use for different tasks:

Use Case	Recommended Framework	Reason
Learning QC basics	Qiskit	Best documentation and tutorials
QML research	PennyLane	Automatic differentiation
IBM hardware access	Qiskit	Native integration

Google hardware access	Cirq	Native integration
Hybrid ML models	PennyLane	PyTorch/TensorFlow integration
NISQ algorithm research	Cirq	Fine-grained control

Table 4: Framework Selection Guide by Use Case.

In practice, many researchers use multiple frameworks. The underlying quantum concepts — qubits, gates, measurements, entanglement — transfer directly across platforms. Learning one framework deeply makes it straightforward to pick up the others.

> ***Try It Yourself***
>
> *Pick one of the three frameworks and implement a GHZ state circuit. The GHZ state is an extension of the Bell state to three qubits:* $|GHZ\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$. *Start with a Hadamard on qubit 0, then apply CNOT gates from qubit 0 to qubit 1 and from qubit 1 to qubit 2. Execute the circuit and verify that you observe only '000' and '111' outcomes.*

3.4 SUMMARY

This unit introduced the three major quantum programming frameworks used in quantum machine learning:

- **Qiskit (IBM)** is a comprehensive SDK with four components — Terra for circuits, Aer for simulation, Ignis for noise characterisation, and Aqua for algorithms. You learned to create QuantumCircuit objects, add gates, execute on simulators, and interpret results. Qiskit's strength lies in its extensive ecosystem and direct access to IBM's quantum hardware.
- **PennyLane (Xanadu)** is a QML-focused framework featuring the QNode pattern for quantum functions. You explored data encoding methods — Basis, Amplitude, and Angle embedding — that are fundamental to quantum machine

learning. PennyLane's automatic differentiation and integration with classical ML libraries make it the preferred choice for variational quantum algorithms.

- **Cirq (Google)** is a NISQ-focused framework using LineQubit objects and Moment-based circuit construction. You implemented quantum teleportation as a practical example demonstrating circuit construction and simulation. Cirq's hardware-aware design provides fine-grained control essential for research on near-term quantum devices.
- All three frameworks share the same underlying quantum concepts — qubits, gates, measurements, and entanglement — but differ in syntax, design philosophy, and intended use cases.
- The **QNode pattern** in PennyLane makes quantum circuits callable like regular Python functions, enabling seamless integration with classical ML workflows.
- **Data encoding** is the bridge between classical data and quantum computation. Basis embedding handles integers, amplitude embedding achieves exponential compression for large feature vectors, and angle embedding maps features directly to qubit rotations.
- **Moment-based construction** in Cirq provides explicit control over the parallelism and timing of quantum operations, which is critical for hardware optimisation.
- The Bell state example implemented across all three frameworks demonstrates how the same quantum algorithm translates into different syntactic paradigms.
- Framework selection depends on your specific needs: Qiskit for learning and IBM hardware, PennyLane for ML tasks, and Cirq for hardware-aware research.
- The ability to write and execute quantum circuits across these frameworks is prerequisite knowledge for all subsequent quantum machine learning implementations covered later in the course.

3.5 MIND MAP

[This page is reserved for the unit mind map]

3.6 GLOSSARY

Aer — Qiskit's high-performance quantum simulator framework providing state vector, QASM, and noise simulation.

Amplitude Embedding — Data encoding method that stores a normalised classical vector as quantum state amplitudes, requiring only $\log_2(N)$ qubits for N features.

Angle Embedding — Data encoding method that encodes each feature as a rotation angle on a separate qubit using rotation gates.

Aqua — Qiskit component providing algorithms for quantum applications in chemistry, optimisation, and machine learning.

Backend — The execution environment for quantum circuits, either a simulator or actual quantum hardware.

Basis Embedding — Data encoding method that represents a classical integer as a computational basis state using its binary representation.

Cirq — Google's Python framework for programming NISQ quantum computers, featuring moment-based circuit construction and hardware-aware design.

GridQubit — Cirq qubit type arranged in a two-dimensional grid, matching the physical layout of superconducting chip architectures like Google's Sycamore.

Ignis — Qiskit framework for quantum error characterisation, mitigation, and verification.

LineQubit — Cirq qubit type arranged in a linear sequence, identified by integer indices, suitable for general-purpose simulation.

Moment — In Cirq, a collection of operations that execute simultaneously on different qubits within the same time slice.

Parameter-Shift Rule — A quantum gradient computation technique used by PennyLane that evaluates the circuit at shifted parameter values to compute exact derivatives.

PennyLane — Xanadu's open-source framework for quantum machine learning with automatic differentiation and classical ML library integration.

Qiskit — IBM's open-source software development kit for quantum computing, comprising Terra, Aer, Ignis, and Aqua components.

QNode — PennyLane's quantum node, a decorated Python function containing quantum operations and measurements that behaves like a differentiable function.

QuantumCircuit — Qiskit's class representing a quantum circuit as a sequence of gates and measurements applied to quantum and classical registers.

Shots — The number of times a quantum circuit is executed and measured to gather statistical distributions of outcomes.

Terra — Qiskit's foundation component for circuit composition, optimisation, transpilation, and backend communication.

Wires — PennyLane terminology for qubits; the parameter specifying which qubits quantum operations act on.

3.7 SELF-ASSESSMENT QUESTIONS

1. In Qiskit's architecture, which component is responsible for high-performance quantum simulation, including state vector and noise model simulation?

    a) Terra

    b) Aer

    c) Ignis

    d) Aqua

2. In Qiskit, why are both a `QuantumRegister` and a `ClassicalRegister` needed in a circuit that performs measurements?

    a) The QuantumRegister stores gate definitions, while the ClassicalRegister stores circuit metadata

    b) The QuantumRegister holds qubits for quantum operations, while the ClassicalRegister stores the classical measurement outcomes

    c) Both registers hold qubits, but the ClassicalRegister uses noisier qubits

d) The ClassicalRegister is only needed when running on real hardware, not on simulators

3. What is the role of the `@qml.qnode` decorator in PennyLane?

   a) It converts a classical Python function into a compiled C++ function for speed

   b) It binds a quantum function to a specific device and enables it to be executed as a quantum computation that returns measurement results

   c) It automatically parallelises the function across multiple CPU cores

   d) It adds error correction to the quantum circuit defined in the function

4. A dataset has a feature vector of 1024 dimensions. Which data encoding method would use the fewest qubits to encode this vector?

   a) Angle Embedding, requiring 1024 qubits

   b) Basis Embedding, requiring 1024 qubits

   c) Amplitude Embedding, requiring 10 qubits

   d) Amplitude Embedding, requiring 1024 qubits

5. In Cirq, what is the difference between `LineQubit` and `GridQubit`?

   a) LineQubit supports more gates than GridQubit

   b) LineQubit arranges qubits in a one-dimensional linear topology, while GridQubit arranges them in a two-dimensional grid that models physical chip layouts

   c) LineQubit is used for simulators only, while GridQubit is used for real hardware only

   d) GridQubit is faster than LineQubit for all operations

6. In Cirq, a "Moment" represents a collection of operations that:

   a) Must all be applied to the same qubit in sequence

   b) Execute simultaneously on different qubits within the same time slice

   c) Are stored in memory but never executed

   d) Represent the total runtime of the quantum program

7. When executing a quantum circuit with `shots=1000`, what does the `shots` parameter control?

a) The number of qubits in the circuit

b) The number of times the circuit is repeatedly executed and measured, producing a statistical distribution of outcomes

c) The number of gates that can be applied before measurement

d) The maximum execution time in milliseconds

8. Why is PennyLane considered particularly well-suited for quantum machine learning compared to Qiskit or Cirq?

   a) It supports more qubits than other frameworks

   b) It provides built-in automatic differentiation of quantum circuits and integrates natively with ML libraries like PyTorch, TensorFlow, and JAX

   c) It is the only framework that can run on real quantum hardware

   d) It has lower simulation noise than other frameworks

9. A researcher needs to encode the classical data vector [0.3, 0.1, 0.7, 0.2] into a quantum circuit using Angle Embedding. How many qubits are required, and what operation is applied to each qubit?

   a) 2 qubits; each qubit receives two amplitude values

   b) 4 qubits; each qubit undergoes a rotation gate parameterised by one feature value

   c) 4 qubits; the entire vector is stored in the amplitudes of the quantum state

   d) 1 qubit; all features are encoded sequentially using time multiplexing

10. In PennyLane, what is the difference between `qml.expval(qml.PauliZ(0))` and `qml.probs(wires=[0, 1])` as return values of a QNode?

    a) `expval` returns the full quantum state, while `probs` returns a single number

    b) `expval` returns the expectation value of the Pauli-Z observable (a scalar), while `probs` returns the probability distribution over all computational basis states of the specified wires

    c) `expval` can only be used on simulators, while `probs` works on real hardware

d) There is no difference; both return the same measurement result in different formats

3.8 TERMINAL QUESTIONS

SHORT-ANSWER QUESTIONS (50-100 WORDS)

1. Describe the four main components of the Qiskit framework (Terra, Aer, Ignis, Aqua) and their respective roles.
2. Explain the QNode pattern in PennyLane. What is a QNode, how is it created, and why is it central to the framework's design?
3. What are the three data encoding methods available in PennyLane (Basis, Amplitude, and Angle embedding)? Briefly describe each and state the number of qubits required.
4. Explain how Cirq's moment-based circuit construction works and how it differs from the sequential approach used by Qiskit.
5. What is automatic differentiation in PennyLane and why is it important for quantum machine learning?

LONG-ANSWER QUESTIONS (150-200 WORDS)

1. Compare the design philosophies of Qiskit, PennyLane, and Cirq in terms of their primary focus areas and intended use cases. Discuss when you would choose each framework.
2. Explain amplitude embedding in detail, including the mathematical formula, qubit requirements, normalisation considerations, and its advantage for encoding large classical datasets into quantum states. Illustrate with a code example.
3. Compare the process of creating a Bell state across Qiskit, PennyLane, and Cirq. Present the code for each framework and discuss how the syntax differences reflect each framework's design philosophy.

4. Describe the complete workflow for building, executing, and interpreting results from a quantum circuit in Qiskit, including circuit creation, backend selection, simulation, and result analysis.
5. Discuss how PennyLane bridges quantum computing and classical machine learning. Explain the role of variational circuits, how trainable parameters are optimised, and how PennyLane integrates with classical ML libraries such as PyTorch and TensorFlow.

3.9 ANSWERS

3.9.1 SELF-ASSESSMENT ANSWERS

1. **(b)** Aer is Qiskit's high-performance simulator framework. It provides state vector simulation for exact results, QASM simulation for shot-based measurement, and noise models for realistic simulation. Terra handles circuit construction, Ignis handles noise characterisation, and Aqua provides algorithm libraries.
2. **(b)** The QuantumRegister holds quantum bits (qubits) on which quantum operations are performed, while the ClassicalRegister stores the classical bits that receive the results of quantum measurements. Measurements collapse qubit states to classical values that must be stored in classical bits.
3. **(b)** The `@qml.qnode(dev)` decorator binds a Python function containing quantum operations to a specific quantum device. It transforms the function into a QNode — PennyLane's core abstraction — that can be executed as a quantum computation, differentiated for gradient-based optimisation, and returns measurement results.
4. **(c)** Amplitude Embedding stores a normalised classical vector as quantum state amplitudes, requiring only $\log_2(1024) = 10$ qubits to encode 1024 features. Angle Embedding would need 1024 qubits (one per feature), and Basis Embedding encodes integers in computational basis states.

5. **(b)** LineQubit arranges qubits in a one-dimensional linear topology, suitable for general-purpose simulation. GridQubit arranges them in a two-dimensional grid that models the physical layout of superconducting quantum chips like Google's Sycamore processor.
6. **(b)** A Moment in Cirq is a collection of operations that execute simultaneously on different qubits within the same time slice. Cirq automatically organises operations into Moments, ensuring that operations on the same qubit are placed in separate Moments while independent operations can share a Moment.
7. **(b)** The `shots` parameter specifies how many times the quantum circuit is repeatedly executed and measured. Since quantum measurement is probabilistic, running multiple shots produces a histogram of outcomes that approximates the theoretical probability distribution.
8. **(b)** PennyLane was designed with machine learning as a primary focus. Its key advantage is built-in automatic differentiation of quantum circuits, which enables gradient-based optimisation of variational circuits. It also integrates natively with major ML frameworks (PyTorch, TensorFlow, JAX), making hybrid quantum-classical models straightforward to build.
9. **(b)** Angle Embedding requires one qubit per feature, so 4 features need 4 qubits. Each feature value is used as the rotation angle of a rotation gate (typically R_Y) applied to the corresponding qubit: $|\psi\rangle = \otimes_j R_Y(x_j)|0\rangle$.
10. **(b)** `qml.expval(qml.PauliZ(0))` returns the expectation value of the Pauli-Z observable on qubit 0, which is a single scalar between -1 and 1. `qml.probs(wires=[0, 1])` returns the probability distribution over all computational basis states of the specified wires (a vector of probabilities summing to 1).

3.9.2 TERMINAL QUESTION ANSWERS

1. Qiskit consists of four main components. **Terra** is the foundation layer that provides tools for composing quantum circuits using the QuantumCircuit class, handling qubit

and gate operations. **Aer** provides high-performance simulators for testing quantum circuits without real hardware, including the qasm*simulator for shot-based simulation and statevector*simulator for exact state computation. **Ignis** focuses on noise characterisation, error mitigation, and quantum hardware calibration. **Aqua** provides high-level algorithm modules for applications in chemistry, optimisation, finance, and machine learning. Together, these components form a comprehensive ecosystem from low-level circuit construction to high-level applications.

2. A QNode (quantum node) is PennyLane's core abstraction — a Python function decorated with `@qml.qnode(dev)` that contains quantum operations and returns measurement results. It is created by writing a function that applies quantum gates and returns measurements, then decorating it with `@qml.qnode` bound to a specific device. The QNode is central because it makes quantum circuits callable like regular Python functions, enables automatic differentiation for gradient computation, and seamlessly integrates quantum computations into classical ML workflows. Parameters can be passed to QNodes, making them ideal for variational quantum circuits with trainable weights.
3. **Basis embedding** encodes a classical integer as a computational basis state ($|x_1 x_2 \dots x_n\rangle$), requiring n qubits for n-bit integers. It is best for integer or categorical data. **Amplitude embedding** encodes a normalised feature vector as quantum state amplitudes ($|\psi\rangle = \sum \hat{x}_i |i\rangle$), requiring only $\log_2(N)$ qubits for N features — an exponential compression. It is ideal for large feature vectors. **Angle embedding** encodes each feature as a rotation angle on a separate qubit ($\bigotimes R_Y(x_j)|0\rangle$), requiring one qubit per feature. It is best suited for small feature sets where each feature maps directly to a qubit rotation.
4. Cirq uses a **moment-based** circuit construction where operations are grouped into "moments" — collections of gates that execute simultaneously on different qubits. Each moment represents a single time step in the circuit. This gives explicit control over operation timing and parallelism. In contrast, Qiskit uses a **sequential** approach where gates are appended to the circuit one at a time and the framework handles

scheduling. Cirq's approach is more hardware-aware, directly reflecting the physical constraints of quantum devices where certain operations can run in parallel while others must be sequential due to qubit connectivity.

5. Automatic differentiation in PennyLane computes gradients of quantum circuits with respect to their parameters without requiring manual derivative calculation. Using techniques like the parameter-shift rule, PennyLane evaluates the gradient by running the circuit at shifted parameter values. This is critical for QML because variational quantum algorithms optimise circuit parameters using gradient descent. Automatic differentiation enables seamless training of quantum models with the same optimisation techniques used in classical deep learning, allowing researchers to focus on circuit design rather than gradient derivation.
6. **Qiskit** (IBM) is a comprehensive, education-focused ecosystem designed for broad quantum computing with extensive tutorials and direct access to IBM quantum hardware. It is the best choice for beginners and for researchers needing IBM's quantum processors. **PennyLane** (Xanadu) is an ML-first framework built specifically for variational quantum algorithms, featuring automatic differentiation and integration with PyTorch, TensorFlow, and JAX. Choose PennyLane when building hybrid quantum-classical ML models or training variational circuits. **Cirq** (Google) is a NISQ-focused, research-oriented framework that provides fine-grained hardware-aware control, used in quantum supremacy experiments. Choose Cirq when working with Google's hardware or when you need precise control over circuit timing and qubit topology. In practice, many researchers use multiple frameworks depending on the task at hand.
7. Amplitude embedding encodes a classical feature vector into the amplitudes of a quantum state, achieving exponential data compression. The mathematical formula is:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \hat{x}_i |i\rangle$$

where $\hat{x}$ is the normalised feature vector ($\|\hat{x}\| = 1$) and $n = \log_2(N)$ qubits encode N features. This is a major advantage: a vector of 2^n features requires only n qubits. For example, 1024 features need just 10 qubits.

The feature vector must be normalised to satisfy the quantum state normalisation constraint. PennyLane's `AmplitudeEmbedding` can handle this automatically with the `normalize=True` parameter. The number of features must equal 2^n (padded with zeros if necessary).

Example code:

```
import pennylane as qml
dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev)
def amplitude_circuit(features):
    qml.AmplitudeEmbedding(features, wires=range(2),
normalize=True)
    return qml.state()

features = [0.5, 0.5, 0.5, 0.5]  # 4 values for 2
qubits
state = amplitude_circuit(features)
```

The primary advantage is exponential compression of classical data, making it feasible to process large datasets on near-term quantum devices. However, the state preparation circuit itself can require $O(2^n)$ gates for arbitrary vectors, which is an active area of research to improve.

8. Creating a Bell state $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ involves applying a Hadamard gate followed by a CNOT gate. Each framework implements this differently.

Qiskit uses an explicit circuit-and-backend model:

```
from qiskit import QuantumCircuit, execute, Aer
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])
```

```
backend = Aer.get_backend('qasm_simulator')
result = execute(qc, backend, shots=1000).result()
counts = result.get_counts()
```

This reflects Qiskit's comprehensive approach: circuits, classical registers, explicit measurement, backend selection, and shot-based execution are all separate, clearly defined steps.

PennyLane uses the QNode functional pattern:

```
import pennylane as qml
dev = qml.device("default.qubit", wires=2)
@qml.qnode(dev)
def bell_state():
    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 1])
    return qml.probs(wires=[0, 1])
probs = bell_state()
```

This reflects PennyLane's ML-first philosophy: circuits are Python functions returning measurements, naturally fitting into optimisation workflows.

Cirq uses moment-based construction:

```
import cirq
q0, q1 = cirq.LineQubit.range(2)
circuit = cirq.Circuit([cirq.H(q0), cirq.CNOT(q0, q1),
cirq.measure(q0, q1, key='m')])
result = cirq.Simulator().run(circuit,
repetitions=1000)
```

This reflects Cirq's hardware-aware, research-oriented design with explicit qubit objects and timing control.

9. The complete Qiskit workflow involves several distinct stages:

Circuit creation: Instantiate a `QuantumCircuit(n_qubits, n_classical_bits)` object. Add quantum gates using methods like `qc.h(qubit)` for Hadamard, `qc.cx(control, target)` for CNOT, and `qc.rx(angle, qubit)` for rotations. Add measurements with

`qc.measure(quantum_reg, classical_reg)` to map qubit outcomes to classical bits.

Backend selection: Choose a simulation or hardware backend. Common simulators include `Aer.get_backend('qasm_simulator')` for shot-based measurement simulation and `Aer.get_backend('statevector_simulator')` for exact state vector computation. For real hardware, `IBMQ.load_account()` provides access to IBM Quantum systems.

Execution: Run the circuit using `execute(circuit, backend, shots=N)`, specifying the number of measurement repetitions. The `result()` method blocks until execution completes.

Result analysis: Extract results using `result.get_counts()`, which returns a dictionary mapping measurement outcome bit strings to their frequency counts. For example, a Bell state measurement might yield `{'00': 498, '11': 502}`. The statevector simulator returns the full state vector via `result.get_statevector()`. Results can be visualised using Qiskit's built-in plotting functions or analysed programmatically for algorithm validation.

10. PennyLane bridges quantum computing and classical ML through three key mechanisms.

Variational circuits are parameterised quantum circuits where gate rotation angles are trainable parameters. A typical variational circuit has three layers: a data encoding layer (embedding classical features into quantum states using angle or amplitude embedding), a variational layer (applying parameterised rotation gates R_X, R_Y, R_Z with trainable weights), and an entangling layer (CNOT gates creating correlations between qubits). The circuit output — typically expectation values of observables — serves as the model's prediction.

Parameter optimisation uses gradient-based methods. PennyLane computes gradients automatically using the parameter-shift rule, which evaluates the circuit at shifted parameter values. The gradient function is obtained via `qml.grad(circuit, argnum=0)`, enabling standard gradient descent. Optimisers like

`qml.GradientDescentOptimizer` or `qml.AdamOptimizer` iteratively update parameters to minimise a cost function, exactly as in classical neural network training.

Classical ML integration is PennyLane's distinguishing feature. QNodes can be embedded directly into PyTorch (`nn.Module`), TensorFlow (`tf.keras.Layer`), or JAX computational graphs. This allows hybrid quantum-classical models where some layers are classical neural network layers and others are quantum circuits. Gradients flow seamlessly through both classical and quantum components, enabling end-to-end training. For example, a PyTorch model might use classical convolutional layers for feature extraction followed by a quantum circuit layer for classification, with backpropagation training the entire pipeline jointly.

3.10 REFERENCES

BOOKS

- Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th Anniversary Edition). Cambridge University Press.
- Pattanayak, S. (2021). *Quantum Machine Learning with Python*. Apress.

JOURNAL ARTICLES

- Du, Y., Hsieh, M.-H., Liu, T., You, S., & Tao, D. (2025). Quantum Machine Learning: A Hands-on Tutorial. *arXiv preprint arXiv:2502.01146*.

WEB REFERENCES

- Cirq Developers. (2024). *Cirq Documentation*. https://quantumai.google/cirq
- IBM Quantum. (2024). *Qiskit Documentation*. https://docs.quantum.ibm.com/
- Xanadu AI. (2024). *PennyLane Documentation*. https://pennylane.ai/qml/